```
    }),
  })

 }

 const seatItems = Array.from(Array(props.seatsPerRow).keys()).map(
  (seatNumber: number) => {
   return (
    <Seat
     clickHandler={onSeatChange}
     key={seatNumber}
     seatNumber={seatNumber}
     status={validSeatStatus(seatNumber)}
    />
   )
  }
 )

 return <tr className="h-20">{seatItems}</tr>
}

export default Row
```

There are some small changes here because we are passing concertId and rowData into the props. There are a few bigger changes as well. First, we have a useEffect hook here to manage the seat status. Each row maintains a local status of seats based on the ticket data, which is updated by a local click. The useEffect hook here regenerates that local status when the row data changes. So, to continue the workflow, if the Venue gets a new set of row data, it will get passed through to this component, and the effect hook will fire and update the status of the seats to reflect the new data. By using useEffect here, we get a chance to manipulate the raw data—in this case converting the entire set of ticket data into an array of seat statuses that we can pass to the setSeatStatuses setter function which was created by the useState hook.

There is a new status that a seat can have: “purchased”, and a few relatively minor changes deal with that new status.

Then another big change in the onSeatChange click handler. We’ve updated this method to make a fetch call back to the server to update data there. We’re